
23 All-Pairs Shortest Paths

In this chapter, we turn to the problem of finding shortest paths between all pairs of vertices in a graph. A classic application of this problem occurs in computing a table of distances between all pairs of cities for a road atlas. Classic perhaps, but not a true application of finding shortest paths between *all* pairs of vertices. After all, a road map modeled as a graph has one vertex for *every* road intersection and one edge wherever a road connects intersections. A table of intercity distances in an atlas might include distances for 100 cities, but the United States has approximately 300,000 signal-controlled intersections¹ and many more uncontrolled intersections.

A legitimate application of all-pairs shortest paths is to determine the *diameter* of a network: the longest of all shortest paths. If a directed graph models a communication network, with the weight of an edge indicating the time required for a message to traverse a communication link, then the diameter gives the longest possible transit time for a message in the network.

As in Chapter 22, the input is a weighted, directed graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$ that maps edges to real-valued weights. Now the goal is to find, for every pair of vertices $u, v \in V$, a shortest (least-weight) path from u to v , where the weight of a path is the sum of the weights of its constituent edges. For the all-pairs problem, the output typically takes a tabular form in which the entry in u 's row and v 's column is the weight of a shortest path from u to v .

You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source. If all edge weights are nonnegative, you can use Dijkstra's algorithm. If you implement the min-priority queue with a linear array, the running time is $O(V^3 + VE)$ which is $O(V^3)$. The binary min-heap implementation of the min-priority queue yields a running time of $O(V(V + E) \lg V)$. If $|E| = \Omega(V)$, the running time becomes $O(VE \lg V)$, which is faster than $O(V^3)$ if the graph is sparse. Alternatively, you can implement the min-priority queue with a Fibonacci heap, yielding a running time of $O(V^2 \lg V + VE)$.

If the graph contains negative-weight edges, Dijkstra's algorithm doesn't work, but you can run the slower Bellman-Ford algorithm once from each vertex. The resulting running time is $O(V^2E)$, which on a dense graph is $O(V^4)$. This chapter shows how to guarantee a much better asymptotic running time. It also investigates the relation of the all-pairs shortest-paths problem to matrix multiplication.

Unlike the single-source algorithms, which assume an adjacency-list representation of the graph, most of the algorithms in this chapter represent the graph by an adjacency matrix. (Johnson's algorithm for sparse graphs, in Section 23.3, uses adjacency lists.) For convenience, we assume that the vertices are numbered $1, 2, \dots, |V|$, so that the input is an $n \times n$ matrix $W = (w_{ij})$ representing the edge weights of an n -vertex directed graph $G = (V, E)$, where

$$w_{ij} = \begin{cases} 0 & \text{if } i = j, \\ \text{the weight of directed edge } (i, j) & \text{if } i \neq j \text{ and } (i, j) \in E, \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E. \end{cases} \quad (23.1)$$

The graph may contain negative-weight edges, but we assume for the time being that the input graph contains no negative-weight cycles.

The tabular output of each of the all-pairs shortest-paths algorithms presented in this chapter is an $n \times n$ matrix. The (i, j) entry of the output matrix contains $\delta(i, j)$, the *shortest-path weight from vertex i to vertex j* , as in Chapter 22.

A full solution to the all-pairs shortest-paths problem includes not only the shortest-path weights but also a *predecessor matrix* $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j , and otherwise π_{ij} is the predecessor of j on some shortest path from i . Just as the predecessor subgraph G_π from Chapter 22 is a shortest-paths tree for a given source vertex, the subgraph induced by the i th row of the Π matrix should be a shortest-paths tree with root i . For each vertex $i \in V$, the *predecessor subgraph* of G for i is $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$, where

$$V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{NIL}\} \cup \{i\},$$

$$E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} - \{i\}\}.$$

If $G_{\pi,i}$ is a shortest-paths tree, then PRINT-ALL-PAIRS-SHORTEST-PATH on the following page, which is a modified version of the PRINT-PATH procedure from Chapter 20, prints a shortest path from vertex i to vertex j .

In order to highlight the essential features of the all-pairs algorithms in this chapter, we won't cover how to compute predecessor matrices and their properties as extensively as we dealt with predecessor subgraphs in Chapter 22. Some of the exercises cover the basics.

PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, j)

```

1  if  $i == j$ 
2      print  $i$ 
3  elseif  $\pi_{ij} == \text{NIL}$ 
4      print "no path from"  $i$  "to"  $j$  "exists"
5  else PRINT-ALL-PAIRS-SHORTEST-PATH( $\Pi, i, \pi_{ij}$ )
6      print  $j$ 
```

Chapter outline

Section 23.1 presents a dynamic-programming algorithm based on matrix multiplication to solve the all-pairs shortest-paths problem. The